

ther be something you know (memorized secrets, passwords, passphrases, PINs), something you are (biometrics, fingerprint, face scans), or something you have (OTP tokens, a cryptographic device such as a smart card).

- **Single Sign-on Authentication (SSO)** –This occurs when a user logs into one application and is then automatically logged into other applications without having to re-authenticate. For example, when logging into Google, the user will be automatically logged into other Google services such as YouTube, Google Docs, and Gmail.
- **Software bill of materials (SBOM)** - A structured, comprehensive list of all components, modules, libraries, frameworks and other resources required to build or assemble a software application.
- **Software Composition Analysis (SCA)** –A set of technologies designed to analyze application composition, dependencies, libraries and packages for security vulnerabilities of specific component versions in use. This is not to be confused with source-code analysis which is now commonly referred to as SAST.
- **Software development lifecycle (SDLC)** –The step-by-step process by which software is developed going from the initial requirements to deployment and maintenance.
- **SQL Injection (SQLi)** –A code injection technique used to attack data-driven applications, in which malicious SQL statements are inserted into an entry point.
- **Stateful Session Mechanism** –In a stateful session mechanism, the application retains session state at the backend which typically corresponds to a session token, generated using a cryptographically secure pseudo-random number generator (CSPRNG), which is issued to the end user.
- **Stateless Session Mechanism** –A stateless session mechanism will use a self-contained token which is passed to clients, and contains session information that is not necessarily stored within the service which then receives and validates the token. In reality, a service will need to have access to some session information (such as a JWT revocation list) in order to be able to enforce required security controls.
- **Static application security testing (SAST)** –A set of technologies designed to analyze application source code, byte code and binaries for coding and design conditions that are indicative of security vulnerabilities. SAST solutions analyze an application from the “inside out” in a non-running state.
- **Threat Modeling** –A technique consisting of developing increasingly refined security architectures to identify threat agents, security zones, security controls, and important technical and business assets.
- **Time-of-check to time-of-use (TOCTOU)** –A situation where an application checks the state of a resource before using that resource, but the resource’s state can be changed between the check and the use. This can invalidate the results of the check and cause a situation where the application performs invalid actions due to this state mismatch.
- **Time based One-time Passwords (TOTPs)** - A method of generating an OTP where the current time acts as part of the algorithm to generate the password.
- **TLS client authentication**, also called **Mutual TLS (mTLS)** –In a standard TLS connection, a

client can use the certificate provided by the server to validate the server's identity. Where TLS client authentication is used, the client also uses its own private key and certificate to allow the server to also validate the client's identity.

- **Transport Layer Security (TLS)** –Cryptographic protocols that provide communication security over a network connection.
- **Traversal Using Relays around NAT (TURN)** –An extension of the STUN protocol using a TURN server as a relay when direct peer-to-peer connections cannot be established. Defined in RFC 8656.
- **Trusted execution environment (TEE)** - An isolated processing environment in which applications can be securely executed irrespective of the rest of the system.
- **Trusted Platform Module (TPM)** –A type of HSM that is usually attached to a larger hardware component such as a motherboard and acts as the “root of trust”for that system.
- **Trusted Service Layer** –Any trusted control enforcement point, such as a microservice, serverless API, server-side, a trusted API on a client device that has secure boot, partner or external APIs, and so on. Trusted means that there is no concern that an untrusted user will be able to bypass or skip the layer or controls implemented at that layer.
- **Uniform Resource Identifier (URI)**- A unique string of characters that identifies a resource, such as webpage, mail address, places.
- **Uniform Resource Locator (URL)** –A string that specifies the location of resource on the Internet.
- **Universally Unique Identifier (UUID)** –A unique reference number used as an identifier in software.
- **Verifier** –The person or team that is reviewing an application against the OWASP ASVS requirements.
- **Web Real-Time Communication (WebRTC)** –A protocol stack and associated web API used for the transport of multimedia streams in web applications, usually in the context of teleconferencing. Based on SRTP, SRTCP, DTLS, SDP and STUN/TURN.
- **WebSocket over TLS (WSS)** –A practice of securing WebSocket communication by layering WebSocket over TLS protocol.
- **What You See Is What You Get (WYSIWYG)** –A type of rich content editor that shows how the content will actually look when rendered rather than showing the coding used to govern the rendering.
- **X.509 Certificate** –An X.509 certificate is a digital certificate that uses the widely accepted international X.509 public key infrastructure (PKI) standard to verify that a public key belongs to the user, computer or service identity contained within the certificate.
- **XML eXternal Entity (XXE)** –A type of XML entity that can access local or remote content via a declared system identifier. This may lead to various injection attacks.

Appendix B: References

The following OWASP projects are most likely to be useful to users/adopters of this standard:

OWASP Core Projects

1. OWASP Top 10 Project: <https://owasp.org/www-project-top-ten/>
2. OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
3. OWASP Proactive Controls: <https://owasp.org/www-project-proactive-controls/>
4. OWASP Software Assurance Maturity Model (SAMM): <https://owasp.org/www-project-samm/>
5. OWASP Secure Headers Project: <https://owasp.org/www-project-secure-headers/>

OWASP Cheat Sheet Series project

This project has several cheat sheets that will be relevant to different topics in the ASVS.

There is a mapping to the ASVS which can be found here: <https://cheatsheetseries.owasp.org/IndexASVS.html>

Mobile Security Related Projects

1. OWASP Mobile Security Project: <https://owasp.org/www-project-mobile-security/>
2. OWASP Mobile Top 10 Risks: <https://owasp.org/www-project-mobile-top-10/>
3. OWASP Mobile Security Testing Guide and Mobile Application Security Verification Standard: <https://owasp.org/www-project-mobile-security-testing-guide/>

OWASP Internet of Things related projects

1. OWASP Internet of Things Project: <https://owasp.org/www-project-internet-of-things/>

OWASP Serverless projects

1. OWASP Serverless Project: <https://owasp.org/www-project-serverless-top-10/>

Others

Similarly, the following websites are most likely to be useful to users/adopters of this standard

1. SecLists Github: <https://github.com/danielmiessler/SecLists>
2. MITRE Common Weakness Enumeration: <https://cwe.mitre.org/>

3. PCI Security Standards Council: <https://www.pcisecuritystandards.org/>
4. PCI Data Security Standard (DSS) v3.2.1 Requirements and Security Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI_DSS_v3-2-1.pdf
5. PCI Software Security Framework - Secure Software Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-Software-Standard-v1_0.pdf
6. PCI Secure Software Lifecycle (Secure SLC) Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-SLC-Standard-v1_0.pdf
7. OWASP ASVS 4.0 Testing Guide <https://github.com/BlazingWind/OWASP-ASVS-4.0-testing-guide>

Appendix C: Cryptography Standards

The “Cryptography” chapter goes beyond simply defining best practices. It aims to enhance understanding of cryptography principles and encourage the adoption of more resilient, modern security methods. This appendix provides detailed technical information regarding each requirement, complementing the overarching standards outlined in the “Cryptography” chapter.

This appendix defines the level of approval for different cryptographic mechanisms:

- Approved (A) mechanisms can be used in applications.
- Legacy mechanisms (L) should not be used in applications but might still be used for compatibility with existing legacy applications or code only. While the usage of such these mechanisms is currently not considered to be a vulnerability in itself, they should be replaced by more secure and future-proof mechanisms as soon as possible.
- Disallowed mechanisms (D) must not be used because they are currently considered broken or do not provide sufficient security.

This list may be overridden in the context of a given application for various reasons including:

- new evolutions in the field of cryptography;
- compliance with regulation.

Cryptographic Inventory and Documentation

This section provides additional information for V11.1 Cryptographic Inventory and Documentation.

It is important to ensure that all cryptographic assets, such as algorithms, keys, and certificates, are regularly discovered, inventoried, and assessed. For Level 3, this should include the use of static and dynamic scanning to discover the use of cryptography in an application. Tools such as SAST and DAST may help with this but it is possible that dedicated tools would be needed to get more comprehensive coverage. Freeware examples of tools include:

- CryptoMon - Network Cryptography Monitor - using eBPF, written in python

- Cryptobom Forge Tool: Generating Comprehensive CBOMs from CodeQL Outputs

Equivalent Strengths of Cryptographic Parameters

The relative security strengths for various cryptographic systems are in this table (from NIST SP 800-57 Part 1, p.71):

Security Strength	Symmetric Key Algorithms	Finite Field	Integer Factorisation	Elliptic Curve
≤ 80	2TDEA	L = 1024 N = 160	k = 1024	f = 160-223
112	3TDEA	L = 2048 N = 224	k = 2048	f = 224-255
128	AES-128	L = 3072 N = 256	k = 3072	f = 256-383
192	AES-192	L = 7680 N = 384	k = 7680	f = 384-511
256	AES-256	L = 15360 N = 512	k = 15360	f = 512+

Example of applications:

- Finite Field Cryptography: DSA, FFDH, MQV
- Integer Factorisation Cryptography: RSA
- Elliptic Curve Cryptography: ECDSA, EdDSA, ECDH, MQV

Note: that this section assumes that no quantum computer exists; if such a computer would exist, the estimates for the last 3 columns would be no longer valid.

Random Values

This section provides additional information for V11.5 Random Values.

Name	Version/Reference	Notes	Status
/dev/random	Linux 4.8+ (Oct 2016), also found in iOS, Android, and other Linux-based POSIX operating systems. Based on RFC7539	Utilizing ChaCha20 stream. Found in iOS SecRandomCopyBytes and Android Secure Random with the correct settings provided to each.	A

Name	Version/Reference	Notes	Status
/dev/urandom	Linux kernel's special file for providing random data	Provides high-quality, entropy sources from hardware randomness	A
AES-CTR-DRBG	NIST SP800-90A	As used in common implementations, such as Windows CNG API BCryptGenRandom set by BCRYPT_RNG_ALGORITHM.	A
HMAC-DRBG	NIST SP800-90A		A
Hash-DRBG	NIST SP800-90A		A
getentropy()	OpenBSD, available in Linux glibc 2.25+ and macOS 10.12+	Provides secure random bytes directly from the kernel's entropy source with a straightforward and minimal API. It's more modern and avoids pitfalls associated with older APIs.	A

The underlying hash function used with HMAC-DRBG or Hash-DRBG must be approved for this usage.

Cipher Algorithms

This section provides additional information for V11.3 Encryption Algorithms.

Approved cipher algorithms are listed in order of preference.

Symmetric Key Algorithms	Reference	Status
AES-256	FIPS 197	A
Salsa20	Salsa 20 specification	A
XChaCha20	XChaCha20 Draft	A
XSalsa20	Extending the Salsa20 nonce	A

Symmetric Key Algorithms	Reference	Status
ChaCha20	RFC 8439	A
AES-192	FIPS 197	A
AES-128	FIPS 197	L
2TDEA		D
TDEA (3DES/3DEA)		D
IDEA		D
RC4		D
Blowfish		D
ARC4		D
DES		D

AES Cipher Modes

Block ciphers, such as AES, can be used with different modes of operations. Many modes of operations, such as Electronic codebook (ECB), are insecure and must not be used. The Galois/Counter Mode (GCM) and Counter with cipher block chaining message authentication code (CCM) modes of operations provide authenticated encryption and should be used in modern applications.

Approved modes are listed in order of preference.

Mode	Authenticated	Reference	Status	Restriction
GCM	Yes	NIST SP 800-38D	A	
CCM	Yes	NIST SP 800-38C	A	
CBC	No	NIST SP 800-38A	L	
CCM-8	Yes		D	
ECB	No		D	
CFB	No		D	
OFB	No		D	
CTR	No		D	

Notes:

- All encrypted messages must be authenticated. For ANY use of CBC mode there MUST be an associated hashing MAC algorithm to validate the message. In general, this MUST be applied in

the Encrypt-Then-Hash method (but TLS 1.2 uses Hash-Then-Encrypt instead). If this cannot be guaranteed, then CBC MUST NOT be used. The only application where encryption without a MAC algorithm is allowed is disk encryption.

- If CBC is used, it shall be guaranteed that the verification of the padding is performed in constant time.
- When using CCM-8, the MAC tag only has 64 bits of security. This does not conform to requirement 6.2.9 which requires at least 128 bits of security.
- Disk encryption is considered out of scope for the ASVS. Therefore this appendix does not list any approved method for disk encryption. For this usage, encryption without authentication is usually accepted and the XTS, XEX and LRW modes are typically used.

Key Wrapping

Cryptographic key wrap (and corresponding key unwrap) is a method of protecting an existing key by encapsulating (i.e., wrapping) it by employing an additional encryption mechanism so that the original key is not obviously exposed, e.g., during a transfer. This additional key used to protect the original key is referred to as the wrap key.

This operation may be performed when it is desirable to protect keys in places deemed untrustworthy, or to send sensitive keys over untrusted networks or within applications. However, serious consideration should be given to understanding the nature (e.g., the identity and the purpose) of the original key prior to committing to a wrap/unwrap procedure as this may have repercussions for both source and target systems/applications in terms of security and especially compliance which may include audit trails of a key's function (e.g., signing) as well as appropriate key storage.

Specifically, AES-256 MUST be used for key wrapping, following NIST SP 800-38F and considering forward-looking provisions against the quantum threat. Cipher modes using AES are the following, in order of preference:

Key Wrapping	Reference	Status
KW	NIST SP 800-38F	A
KWP	NIST SP 800-38F	A

AES-192 and AES-128 MAY be used if the use case demands it, but its motivation MUST be documented in the entity's cryptography inventory.

Authenticated Encryption

With the exception of disk encryption, encrypted data must be protected against unauthorized modification using some form of authenticated encryption (AE) scheme, usually using an authenticated encryption with associated data (AEAD) scheme.

The application should preferably use an approved AEAD scheme. It might alternatively combine an approved cipher scheme and an approved MAC algorithm with a Encrypt-then-MAC construct.

MAC-then-encrypt is still allowed for compatibility with legacy applications. It is used in TLS v1.2 with old ciphers suites.

AEAD mechanism	Reference	Status
AES-GCM	SP 800-38D	A
AES-CCM	SP 800-38C	A
ChaCha-Poly1305	RFC 7539	A
AEGIS-256	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128L	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
Encrypt-then-MAC		A
MAC-then-encrypt		L

Hash Functions

This section provides additional information for V11.4 Hashing and Hash-based Functions.

Hash Functions for General Use Cases

The following table lists hash functions approved in general cryptographic use cases such as digital signatures:

- Approved hash functions provide strong collision resistance and are suitable for high-security applications.
- Some of these algorithms offer strong resistance to attacks when used with proper cryptographic key management, and so are additionally approved for HMAC, KDF, and RBG functions.

- Hash function with less than 254 bit of output have insufficient collision resistance and must not be used for digital signature or other applications requiring collision resistance. For other usages, they might be used for compatibility and verification ONLY with legacy systems but must not be used in new designs.

Hash function	Reference	Status	Restrictions
SHA3-512	FIPS 202	A	
SHA-512	FIPS 180-4	A	
SHA3-384	FIPS 202	A	
SHA-384	FIPS 180-4	A	
SHA3-256	FIPS 202	A	
SHA-512/256	FIPS 180-4	A	
SHA-256	FIPS 180-4	A	
SHAKE256	FIPS 202	A	
BLAKE2s	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE2b	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE3	BLAKE3 one function, fast everywhere	A	
SHA-224	FIPS 180-4	L	Not suitable for HMAC, KDF, RBG, digital signatures